

Executive Summary

We propose a compliance-focused plan to harvest public charts and playlist data for stream estimation. Key official sources include Spotify (Web API and charts site), YouTube (Data API/trending), Billboard (charts pages), Apple Music/iTunes (RSS/API charts), Shazam (charts pages), Deezer (API charts), and ancillary sources (Last.fm charts, etc.). Each source is assessed for accessible endpoints, data content, update schedule, and legal terms. Where possible we favor official APIs (Spotify, YouTube, Deezer, Last.fm, Apple RSS) and avoid scraping content that terms explicitly forbid (Billboard, Apple, Spotify, etc.) ¹ ² . For streaming estimates, we will use industry conversions (150 streams = 1 single, 1,500 = 1 album ³ ⁴) and market-share ratios (Apple $\approx$ 40% of Spotify streams, Amazon $\approx$ 35% ⁵ , Tidal $\approx$ few %) to estimate Apple, Amazon, Tidal streams. The plan includes defining database schemas, scheduling APScheduler jobs (cron-like) for each source, and building caching/backoff to respect rate limits and robots.txt. We flag “High Risk” sources (Billboard, general Spotify/Apple content) and emphasize using only user-authorized data via APIs.

Below we detail the source inventory, legal constraints, technical approaches, and a phased implementation roadmap. Tables at end compare sources (API availability, scraping allowance, update freq., data fields, legal risk). Two mermaid diagrams depict the timeline and data-pipeline flowchart.

1. Data Source Inventory

We identify each major public chart or playlist source, its access methods, and key data. The table below and descriptions detail URLs, APIs, content, and freshness.

- **Spotify (Global/Regional Charts & Playlists)** – *API:* Yes. Spotify provides a Web API (OAuth) to read playlists and track info, but **no public “charts” endpoint**. There is an official charts site (charts.spotify.com) and old `spotifycharts.com/regional` pages, but these require login or have been deprecated. Instead, for charts we may rely on Spotify-generated “Top” playlists (e.g. Top 50 USA) which can be accessed as playlists via the API. For playlists, use `GET https://api.spotify.com/v1/users/{user_id}/playlists` or `GET /v1/me/playlists` (scopes: *playlist-read-private*, *playlist-read-collaborative*) and then `GET /v1/playlists/{playlist_id}/tracks` to list tracks and artists. Each track object has a `popularity` metric (0–100) but no raw play count. Playlists update in near-real-time (as users modify them); Spotify global charts playlists are updated daily/weekly (country-specific charts update daily or weekly depending on chart type).
- *Data:* track name/ID, artist, album, playlist position; no raw streams from API. We will extract user-imported playlists (e.g. artist’s own playlist) plus any Spotify “Top 50” charts playlists.
- *API exists:* Yes (Spotify Web API).
- *Sample endpoint:* `GET /v1/playlists/{id}/tracks` (Auth bearer, Spotify OAuth token).
- *Update freq:* Daily for Top-50 playlists; real-time for user playlists.
- *Official API docs:* [Spotify API Reference](#) (e.g. Playlists) ⁶ ⁷ .
- **YouTube (Charts/Trending & Playlists)** – *API:* Yes. Use the YouTube Data API v3. Charts include “trending” videos and Music charts available via API. For trending tracks, use `GET https://`

`www.googleapis.com/youtube/v3/videos?`

`part=snippet,statistics&chart=mostPopular®ionCode=US&videoCategoryId=10`

(Music category) ⁸. For user playlists, use `GET /playlistItems` (needs OAuth scopes like *youtube.readonly* if private, or API key for public playlists). Data includes video ID, title, channel, view count, like count, etc.

- *Data:* video (song) title, artist/channel, view count, upload date, trending rank.
- *API exists:* Yes (YouTube Data API v3).
- *Sample endpoint:* `GET https://www.googleapis.com/youtube/v3/videos?part=snippet,statistics&chart=mostPopular®ionCode=US&maxResults=50&key=API_KEY`.
- *Update freq:* Hourly/daily for trending chart (varies by region), playlists updated as users edit or hourly for official charts.
- *Official docs:* Google Developers (YouTube API Terms prohibit unauthorized scraping) ⁸.

- **Billboard (e.g. Hot 100)** – *API:* No. Billboard's charts (Hot 100, Top 200, genre charts, etc.) are published on [billboard.com/charts/...]. They are updated weekly (new charts every Saturday). We must scrape HTML if used. A typical URL is `https://www.billboard.com/charts/hot-100/YYYY-MM-DD`. Data on the page includes rank, song title, artist, and for Hot 100 additional stats (streams, sales) shown per entry. For example, one can parse HTML elements like `li.o-chart-results-list__item h3.c-title` (title) and sibling spans for artist/streams ⁹.

- *Data:* rank, track title, artist, label, and often stream/sales count on page.
- *Update freq:* Weekly (charts date-stamped).
- *Sample page:* *Scraping only*.
- *Official API:* None. Billboard explicitly forbids automated scraping ("you will not monitor, scrape, index or copy ... any material on the Site by means of any robot, spider or other automatic device..." ¹).
- *Note: High risk;* scraping violates Billboard's terms (via its Terms of Use) ¹. Use only if user provides data or via a licensed feed.

- **Apple Music / iTunes Charts** – *API:* *Partial*. Apple offers iTunes RSS feeds as a quasi-API (no auth) for top songs/albums by country. Example RSS (JSON) URL: `https://rss.itunes.apple.com/api/v1/us/itunes-music/top-songs/all/100/explicit.json`. This returns the Top 100 songs in the US iTunes Store (updated daily). Apple's MusicKit API (Apple Music API) can fetch charts too (but requires Apple dev token). RSS feeds require no key and update daily/weekly. Alternatively, Apple's iTunes Search API can query specific track/album info but not top charts in one call.

- *Data:* rank, track name, artist, album, artwork. No raw streams.
- *Update freq:* Daily (charts refresh ~weekly/daily on iTunes).
- *API exists:* Yes – iTunes RSS (public) and Apple Music API (requires key).
- *Sample endpoint:* (RSS) `GET https://rss.itunes.apple.com/api/v1/us/itunes-music/top-songs/all/100/explicit.json`.
- *Policy:* Apple's terms forbid any scraping/crawling of their content ²; using their official RSS is encouraged and allowed.

- **Shazam Charts** – *API: No*. Shazam (Apple-owned) publishes global and country-specific Top 100 charts at `https://www.shazam.com/charts/top-100/{country}`. The charts list song title, artist, and Shazam index. No public API is offered. We must scrape the HTML or intercept their JSON API (likely at endpoints under `/shazam/v1/`). The robots.txt allows crawling of `/charts/` but Apple's terms forbid automated scraping of any service content ².
- *Data*: rank, title, artist, weekly Shazam count.
- *Update freq*: Weekly or daily (Shazam updates weekly and has a daily "global viral" chart).
- *Sample page*: e.g. `https://www.shazam.com/charts/top-100/united-states/daily`.
- *Official API*: None.
- *Risk*: High (no API, and Apple's general scraper prohibition applies) ².
- **Deezer Charts** – *API: Yes*. Deezer provides a public API with chart endpoints. For global top: `GET https://api.deezer.com/chart` returns an object with "tracks", "albums", "artists", "playlists" – top 50 of each. For country-specific charts: `GET /chart/{countryId}` (e.g. `chart/52` for USA). These require no auth and support JSON.
- *Data*: top tracks (with Deezer track IDs), top artists, etc., plus track info (title, artist, album, position).
- *Update freq*: Daily (Deezer chart is updated daily).
- *Sample endpoint*: `GET https://api.deezer.com/chart/` (global) or `GET /chart/72` (country code).
- *Docs*: Official [Deezer API](#) (supports charts via playlist endpoints) ¹⁰.
- **Last.fm Charts (Global & Geo)** – *API: Yes*. Last.fm's `chart.getTopTracks` (global top tracks) and `geo.getTopTracks` (country top tracks) methods allow free access (API key required). They return track names and playcounts for top tracks. For example: `http://ws.audioscrobbler.com/2.0/?method=chart.gettoptracks&api_key=KEY&format=json`.
- *Data*: track, artist, playcount, rank.
- *Update freq*: Weekly (charts cover last week) or daily depends on call.
- *Docs*: [Last.fm API](#) (see chart methods) ¹¹.
- **Amazon Music** – *API: No (public)*. Amazon Music Unlimited has "Top Hits" playlists, but no open API. A deprecated **Amazon Music Device API** exists (for Alexa) but requires user account and is not for public use. Scraping `music.amazon.com` is possible only for HTML (and requires login for Prime tracks). We deem this **not implementable** in free plan.
- *Data*: (theoretical) top track and artist lists from Amazon's web interface.
- *Risk*: Very high (no official access, likely disallowed by Amazon's terms). We **omit** Amazon in MVP.
- **Other / Regional Charts**: We note the existence of country-specific charts (e.g. UK Official Charts, ARIA Australia, etc.), satellite radio charts, etc. These lack accessible APIs or stable structures. We will not target them initially beyond global charts.

2. Legal & Policy Constraints

We summarize each source's Terms of Service, robots.txt, and API policy:

- **Spotify:** Developer agreement prohibits redistributing Spotify content. Their Web API is meant for personal or user-authorized use; data “may not be transferred to another service” except to enable user data portability ¹². Scraping or republishing charts without API is likely prohibited. **Key points:** Use only the official API with correct OAuth scopes. Do *not* scrape Spotify's charts site. The Developer Policies forbid bypassing the API or caching Spotify's audio content ¹². Also, recent 2026 updates require using a Premium account for development apps (see next section).
- **YouTube:** The YouTube Terms of Service forbid unauthorized downloading or scraping of content ⁸. Only use the official Data API. The robots.txt disallows many user pages but does allow crawling of non-UI endpoints; however, we will rely on the API to stay compliant. Rate limits and quota must be obeyed (10k units/day by default).
- **Billboard:** Their site **Terms of Use explicitly ban any robot/spider copying site data** (as one user noted: “you will not ... scrape, index or otherwise copy any of the material on the Site by means of any robot, spider or other automatic device” ¹). Robots.txt is unverified, but the TOS clearly forbids scraping. **Conclusion:** do not scrape Billboard Hot 100 or other charts. We flag Billboard as **legal risk: High**. If we need chart data, we must obtain it by other means (e.g. third-party API or user entry).
- **Apple (iTunes, Apple Music, Shazam):** Apple's Media Services Terms forbid any bot/script to manipulate or scrape content ². Specifically, “You may not use any ... automated process ... to scrape, copy, or perform analysis of any portion of the Content or Services.” This covers iTunes/Apple Music and Shazam charts. Their robots.txt is not user-friendly for HTML charts. However, Apple *does* provide official RSS feeds for iTunes charts (meant for distribution) which are exempt. Shazam (as an Apple service) has no API, so scraping its site violates Apple's general rules ². **Conclusion:** Only use Apple's official feeds/APIs; do not scrape Apple websites or Shazam's web pages.
- **Deezer:** Their Developer Terms (not easily readable) allow API usage. They have explicit chart API endpoints. We treat the Deezer API as allowed.
- **Last.fm:** Last.fm's API Terms allow free data access; daily limits apply. Use it with their API key, respecting rate limits.
- **Amazon:** Their Music policies (and general Amazon TOS) forbid scraping or using unapproved APIs. We will not scrape Amazon's site.
- **General Practice:** We will always set a respectful User-Agent, observe robots.txt (e.g. not crawling disallowed paths), use exponential backoff on 429/5xx errors, and implement caching to avoid redundant fetches. Data will be accessed at intervals matching update frequency to limit load (e.g. once per week for weekly charts).

3. Technical Feasibility & Approach

For each source, we recommend the safest method:

- **Spotify Web API (Official):** Use OAuth 2.0 (Authorization Code) with scopes `playlist-read-private` and `playlist-read-collaborative` (to access user's private/collaborative playlists) ⁶ ⁷. Ensure the app is set to *Production* (not Development Mode) to avoid 5-user limit and new dev restrictions. Required endpoints:
- `GET /v1/me/playlists?limit=50` (and paginating with `offset`) to list playlists,

- GET `/v1/playlists/{playlist_id}/tracks` (limit=100 per request, paginate via `offset`) to get tracks,
- GET `/v1/tracks/{id}` for details if needed.
- The Authorization header `Bearer {token}` is needed. The token comes from user login. Rate limiting: typically ~10 req/s. If using multiple track IDs, note that bulk endpoints (`/tracks?ids=`) are now deprecated (post-Feb2026). New guidance: fetch individual tracks in parallel if needed ¹³.
- **Example call:**

```
GET https://api.spotify.com/v1/playlists/37i9dQZEVXbKj23U1GF4IR/tracks?
limit=50&offset=0
Authorization: Bearer {ACCESS_TOKEN}
```

- We will store track IDs and metadata via the API, not screen-scrape any Spotify pages.
- **YouTube Data API:** Obtain an API key or OAuth credentials (API key suffices for public data). Use endpoints:
- **Trending videos:** `videos.list` with `chart=mostPopular`. For example:
GET `https://www.googleapis.com/youtube/v3/videos?part=snippet,statistics&chart=mostPopular®ionCode=US&videoCategoryId=10&maxResults=50&key=`
Returns a list of popular music videos with view counts, etc. Paginate if >50.
- **Playlist items:** `playlistItems.list` with `playlistId={playlistId}` to get a user's playlist tracks (if integrating user's YouTube playlists). Requires OAuth if playlist is private (`youtube.readonly` scope), or only API key if public.
- Handle quotas: e.g. `videos.list` (with stats) costs ~4 units per call; 10k quota/day is sufficient for hundreds of calls/day. Implement checks for `quotaExceeded` errors.

- **Example:**

```
GET https://www.googleapis.com/youtube/v3/videos?
part=snippet,statistics&chart=mostPopular&regionCode=US&maxResults=50&key={API_KEY}
```

- **Billboard (HTML scraping – Not Recommended):** Technically one can fetch Billboard pages with `requests` and parse with BeautifulSoup. For example:

```
res = requests.get("https://www.billboard.com/charts/hot-100")
soup = BeautifulSoup(res.text, 'html.parser')
titles = [tag.get_text(strip=True) for tag in soup.select('li.o-chart-
results-list__item h3.c-title')]
artists = [tag.get_text(strip=True) for tag in soup.select('li.o-chart-
results-list__item span.c-label')]
```

These selectors match elements in their HTML ⁹. However, **Billboard's TOS forbid this scraping** ¹. We flag this as *only an emergency fallback* if data is unavailable; prefer user-provided playlist data instead.

- **Apple/iTunes (RSS feeds):** Poll the iTunes RSS endpoints. No auth needed. Example request:

```
GET https://rss.itunes.apple.com/api/v1/us/itunes-music/top-songs/all/100/
explicit.json
```

The JSON response has fields: `id`, `name`, `artistName`, etc. We can parse rank by array position or explicit rank fields. Poll daily or weekly. Rate-limit: minimal (allowed in Apple's developer terms if just fetching RSS).

- **Shazam (Web scraping):** Fetch chart pages or underlying JSON: e.g. `https://www.shazam.com/charts/top-100/united-states`. The HTML can be parsed (song items are in `.chart-top-tracks__item`). Alternatively, intercept the network call the site makes (often a Shazam API). **Caution:** Shazam's site may block scrapers and Apple prohibits automated scraping ². If used, fetch modestly (once per week) and respect robots (they disallow certain API paths but allow main content ¹⁴). Example: parse title via a selector like `.title` on each chart item. But emphasize: use only if absolutely needed and watch for changes.

- **Deezer API:** Use `GET /chart` (and `GET /chart/{countryId}`) endpoints. No auth needed. Example:

```
GET https://api.deezer.com/chart/
```

Returns JSON: e.g. `data.chart.tracks.data` is an array of track objects (id, title, artist). We parse rank by order. No pagination needed (it's fixed size). We could also call `/chart/XX` for specific countries. No quota known, but limit calls to once per day.

- **Last.fm API:** Use API methods with an API key (free to register). For global:

```
GET http://ws.audioscrobbler.com/2.0/?
method=chart.gettoptracks&api_key=KEY&format=json
```

or per country:

```
GET http://ws.audioscrobbler.com/2.0/?
method=geo.gettoptracks&country=US&api_key=KEY&format=json
```

Parse `track.name`, `track.artist`, `playcount`. Updated weekly. Handle JSON parsing.

- **Amazon Music:** *No safe method.* We omit.

In all scraping cases, use a low request rate (e.g. one chart page per minute) and obey robots.txt. For APIs, batch requests efficiently and back off on HTTP 429/500s. Use meaningful `User-Agent` headers, and respect any provided `Retry-After`. We log each fetch (source, time, status) for audit.

4. Gaps & Required API Scopes

Spotify: Ensure the connected app has the following OAuth scopes (via Spotify Developer Dashboard):

- `playlist-read-private` (read user's private playlists) ⁶,
- `playlist-read-collaborative` (include collaborative playlists) ⁷,
- Optionally `user-top-read` (if retrieving user's top tracks) or `user-library-read` (for saved tracks).

Also verify "Premium Required" status: Spotify's Feb 2026 changes mandate that the *app owner* must have a Spotify Premium account ¹⁵ and that some endpoints (batch calls) are disabled ¹³. Thus test that the app is either in Production mode or meets these new dev-mode rules.

YouTube: Enable **YouTube Data API v3** in Google Cloud Console and generate an API key (or OAuth client if user channel data is needed). If using user-specific playlists (private), add OAuth scope `youtube.readonly`. Otherwise, public data (trending charts) only needs an API key. No other special settings required.

Additional APIs: We should also obtain keys for **Last.fm API** (free developer key) and verify **Deezer API** (no key needed, but whitelist our server's IP if required for heavy use).

Currently we plan **no paid APIs**. All sources listed above are free.

5. Data Model & Ingestion Plan

We recommend a normalized database design (Postgres) with the following entities:

- **Tracks:** (id, title, artist, album, Spotify_ID, YouTube_ID, etc) – unify tracks across sources. Use the existing catalog id if matched, else new entry.
- **Playlists/Charts:** (id, source, name, country, type) – represent a chart or user playlist (e.g. "Hot 100", "Top 50 US Spotify").
- **ChartEntries:** (chart_id, track_id, date, rank, streams, plays, extra_info) – records rank history. Fields: e.g. `streams` (if scraped, like Billboard's stream count), `plays` (e.g. YouTube views), `shazamCount`, etc.
- **PlaylistsTracks:** (playlist_id, track_id, added_date, position) – for user playlists.
- **Users/Artists:** (id, name) linking imported playlists to user/artist accounts.

Polling & Cadence: Schedule background jobs via APScheduler (Replit). Example schedule:

- *Hourly:* YouTube trending (to catch daily top changes) and Last.fm (if needed for frequent updates).
- *Daily:* Spotify playlists (or more often if heavy use), Apple RSS, Shazam daily viral chart.
- *Weekly (on chart-day):* Billboard Hot 100 (if used at all), main Shazam Top 100, Deezer global charts.
- *Ad-hoc:* Any user-triggered playlist import from Spotify or YouTube can fetch on demand.

Each run should **deduplicate**: check if a (chart, date, track) tuple already exists before inserting. We'll keep the history of ranks over time. E.g., each week's Billboard Hot 100 yields up to 100 ChartEntries.

Ingestion Steps: For each source job:

1. Fetch data (API or HTML).
2. Parse into rows of (chart, date, track title, artist, metrics).
3. For each track, attempt to **map to our catalog** (using exact match on title+artist, or via Spotify/YouTube search if missing). Create a track record if new.
4. Insert new ChartEntries with rank and any count fields.

Storage & Scaling: Replit with Postgres can handle moderate volumes (few thousand records/week). Prune or archive old entries if needed. Implement caching (e.g. store last fetched date per chart) to skip repeats.

6. Stream Estimation Methodology

To estimate streams on other platforms and sales equivalents, we use standard industry ratios:

- **RIAA Conversions:** 1 track sale = 150 streams; 1 album sale = 1,500 streams ³ ⁴. Therefore we compute "album-equivalent units" or "track units" by dividing streams by these factors. Clearly label these as "estimate (RIAA-based)".

- **Cross-platform ratios:** Using global streaming market shares (2026 data ⁵), we estimate Apple, Amazon, Tidal streams from Spotify counts. For example, Apple's share is ~12.6% vs Spotify 31.7% ($\approx 0.40\times$), Amazon ~11.1% ($\approx 0.35\times$) of Spotify's, YouTube Music ~9.7% ($\approx 0.31\times$). Tidal's share is very small (~1-2%), so we multiply Spotify streams by ~0.03 for Tidal (very approximate). The output should clearly note high uncertainty. We label these as "estimated" values. For example:

- Estimated Apple Music Streams = SpotifyStreams $\times (12.6/31.7) \approx \times 0.40$ ⁵.

- Estimated Amazon Music Streams $\approx$ SpotifyStreams $\times 0.35$.

- Estimated Tidal Streams $\approx$ SpotifyStreams $\times 0.02$.

- YouTube Music (if needed) $\approx$ Spotify $\times 0.30$.

These ratios are rough; we surface a confidence range ($\pm 20\%$ or more) in the UI.

Finally, we convert total streams into *album-equivalent* or *single-equivalent* units:

- AlbumUnits = TotalStreams / 1500 (per RIAA) ⁴,

- SingleUnits = TotalStreams / 150 ³.

7. Compliance & Operational Practices

To remain compliant, we will:

- **Rate-Limit & Backoff:** For each API, respect documented limits. Use exponential backoff on HTTP 429/503. E.g., for Spotify API 429 responses, honor the `Retry-After` header. For YouTube, monitor quota usage and stop if near limit. For any HTML source, delay requests ($>1s$) and never hammer the site.
- **Robots.txt and Terms:** Before any scraping, check the site's robots.txt. E.g., we will not attempt to scrape any path disallowed. We will programmatically skip disallowed charts (e.g. if robots.txt forbids `/charts`). And we only scrape where terms do not explicitly forbid it. Primarily, we rely on APIs to avoid any conflict.

- **User-Agent & Headers:** Set a descriptive User-Agent string. Use TLS/HTTPS for all calls.
- **IP Rotation/Caching:** If scraping becomes heavy, we may route requests through a pool of IPs or proxies to avoid blocking, but initial plan is low-volume enough not to require it. Caching at least daily results prevents repeated fetches.
- **Logging & Audit Trail:** Log every fetch attempt (timestamp, URL, success/error, response code). Keep a history for compliance review or to respond to takedown requests. The logs serve as an audit.
- **Data Privacy:** We only collect public/permissible data. No personal user data beyond what a user explicitly provided (their own playlists). We will not retain any sensitive info beyond what is needed for credits (e.g. artist names). All compliance rules from source terms are included above to avoid legal issues.

8. Implementation Roadmap

Phase 1 (Weeks 0–2): Requirements & Setup – Review architecture, set up database schema (Tracks, Charts, Entries tables). Configure APScheduler in Replit for background jobs. Obtain necessary API credentials (Spotify client ID/secret, YouTube API key, Last.fm API key). Update Spotify app scopes. (Mermaid timeline below.)

Phase 2 (Weeks 2–4): Core API Integration –

- *Spotify:* Test Spotify OAuth and reading a sample playlist. Ensure `playlist-read-private` and `playlist-read-collaborative` are active. Verify app in Production mode (or Dev mode Premium user).
- *YouTube:* Call Data API for trending videos. Write code to parse video list.
- *Apple iTunes RSS:* Fetch Top Songs RSS JSON for several countries to confirm parsing logic.

Phase 3 (Weeks 4–6): Additional Sources –

- *Deezer:* Implement calls to `/chart` endpoint, parse top tracks.
- *Last.fm:* Integrate `chart.getTopTracks` and `geo.getTopTracks` for global and a default country (e.g. US).
- *Billboard Shazam (if needed):* We will skip Billboard to avoid legal risk. For Shazam, optionally parse their public charts HTML minimally (daily charts).
- *Data mapping:* Implement basic track matching (title+artist normalization). Populate the Tracks table.

Phase 4 (Weeks 6–8): Scheduling & Testing –

- Schedule APScheduler jobs for each source (as per update cadence).
- Implement deduplication logic (skip if entry exists).
- Build unit tests: e.g., fetch 2 sources and confirm DB writes.
- Monitor logs for errors and fix parsing issues.

Phase 5 (Weeks 8–10): UI and Calculations –

- Compute streams estimates using RIAA and ratios. Present on the artist's "credits" page (no live streaming needed).
- Design output (JSON or similar) for shareable graphics (as Muso did).
- Ensure conversion button (streams→units) works with the chosen ratios.

Phase 6 (Weeks 10–12): Verification & Launch –

- Perform end-to-end tests for a few artists.

- Verify compliance: double-check logs, user agent behavior, and licensing warnings.
- Finalize monitoring (alert on failures).

Minimum Viable Sources: Start with **Spotify (already connected)**, **YouTube**, **Apple RSS**, **Deezer**, and **Last.fm** – all free and safe. Optionally add Shazam (if needed) but beware terms. Skip Billboard and Amazon until a future licensing deal.

Developer Settings to Verify: In Spotify Dev Dashboard, ensure “App not in Dev Mode” or otherwise Premium-enabled (to avoid 5-user cap ¹⁵). In Google Console, enable YouTube API and set quotas. In Last.fm account, generate API key.

Test & Monitoring: Each background job should log success/failure; we will periodically audit these. Alerts if any source’s format changes or if quotas are hit. Maintain a dashboard of last-run timestamps.

9. Summary Tables and Diagrams

Source	API	Scraping Allowed?	Update Freq.	Key Fields	Legal Risk
Spotify (API)	Yes	No (use API only)	Daily (playlists)	Track ID, title, artist, playlist ID, track popularity	Low (API compliance needed)
Spotify Charts	No	No (forbidden by policy)	Daily/weekly	(Rank, track, artist) [charts site requires login]	High (disallowed)
YouTube	Yes	No (use API only)	Hourly/Daily	Video ID, title, channel (artist), views, likes, date	Low (with API)
Billboard	No	No (explicitly forbidden)	Weekly	Rank, track, artist, streams, sales (if scraped from site)	Very High (forbidden scraping) ¹
Apple/iTunes	Yes (RSS)	Partial (RSS ok, UI no)	Daily/Weekly	Rank, track title, artist, album, cover art	High if scraped; use RSS ²
Shazam	No	No (per Apple TOS)	Weekly/Daily	Rank, track, artist, Shazam count	High (no API, Apple forbids) ²
Deezer	Yes	n/a (API)	Daily	Track title, artist, position	Low (API provided)
Last.fm	Yes	n/a (API)	Weekly	Track title, artist, playcount	Low (open API)
Amazon Music	No	No (no public API)	n/a	– (theoretical playlist data)	Very High (no access)

Mermaid Timeline: High-level rollout schedule

```

ganttt
  dateFormat YYYY-MM-DD
  title      Chart Data Ingestion Timeline
  section Phase 1: Setup
  Requirements & DB schema      :done,    a1, 2026-03-01, 2w
  Obtain API keys & OAuth setup :done,    a2, after a1, 1w
  section Phase 2: Integrations
  Spotify API integration      :active,   a3, after a2, 1.5w
  YouTube API integration      :         a4, after a3, 1.5w
  Apple RSS integration        :         a5, after a3, 1w
  section Phase 3: Additional Sources
  Deezer API integration       :         a6, after a4, 1w
  Last.fm API integration      :         a7, after a4, 1w
  (Optional) Shazam scraping   :         a8, after a6, 1w
  section Phase 4: Testing & Scheduling
  Write ingestion jobs & tests :         a9, after a7, 2w
  APScheduler job scheduling   :         a10, after a9, 1w
  section Phase 5: Output & Launch
  UI/Estimation features       :         a11, after a10, 1.5w
  Final tests & monitoring setup :        a12, after a11, 1w

```

Mermaid Flowchart: Data ingestion pipeline

```

flowchart TD
  subgraph Sources
    A[Spotify Playlists (API)]
    B[YouTube Trending (API)]
    C[Apple iTunes RSS]
    D[Deezer Charts (API)]
    E[Last.fm Charts (API)]
    F[Shazam Charts (Scrape)]
  end
  subgraph Ingestion
    A --> Parser1[Parse Spotify JSON]
    B --> Parser2[Parse YouTube JSON]
    C --> Parser3[Parse Apple RSS]
    D --> Parser4[Parse Deezer JSON]
    E --> Parser5[Parse Last.fm JSON]
    F --> Parser6[Parse Shazam HTML]
  end
  subgraph Transform & Store
    Parser1 --> Normalize[Track Matching & Normalize]
    Parser2 --> Normalize
    Parser3 --> Normalize
    Parser4 --> Normalize
    Parser5 --> Normalize
  end

```

```

Parser6 --> Normalize
Normalize --> DB[(Database)]
Compute[Compute Estimates (RIAA, Market Share)] --> DB
end
DB --> UI["Display Credits & Charts"]

```

Figure: Multiple source feeds (API or scrape) are parsed, normalized (track lookup), and stored in Postgres. A calculation step applies RIAA conversion and market-share ratios to compute estimated streams/units, which feed into the artist credits page.

Sources: Official documentation, terms, and industry reports cited above ³ ⁴ ⁵ ¹ ⁸ ² ¹¹. These guide our choices of API vs. scraping, legal risk assessments, and streaming conversion factors.

¹ Is my first project realistic? : r/learnpython

https://www.reddit.com/r/learnpython/comments/ucna9j/is_my_first_project_realistic/

² Legal - Apple Media Services - Apple

<https://www.apple.com/legal/internet-services/itunes/>

³ ⁴ RIAA Gold & Platinum Program - RIAA

<https://www.riaa.com/gold-platinum/about-awards/>

⁵ Music Streaming Services Stats (2026)

<https://explodingtopics.com/blog/music-streaming-stats>

⁶ ⁷ Scopes | Spotify for Developers

<https://developer.spotify.com/documentation/web-api/concepts/scopes>

⁸ Your content & third-party training - YouTube Help

<https://support.google.com/youtube/answer/15509945?hl=en>

⁹ Methods to Scrape Billboard and Spotify Data Using Python

<https://www.ottscrape.com/best-methods-scrape-billboard-spotify-data-with-python.php>

¹⁰ Create a Cool Music Chart Site with Deezer API (No Backend Needed!) - DEV Community

<https://dev.to/reynaldi/create-a-cool-music-chart-site-with-deezer-api-no-backend-needed-386e>

¹¹ API Docs | Last.fm

<https://www.last.fm/api>

¹² Web API Reference | Spotify for Developers

<https://developer.spotify.com/documentation/web-api/reference/get-featured-playlists>

¹³ ¹⁵ February 2026 Web API Dev Mode Changes - Migration Guide | Spotify for Developers

<https://developer.spotify.com/documentation/web-api/tutorials/february-2026-migration-guide>

¹⁴ Robots.txt for shazam.com

<https://konigle.com/free-tools/robots-txt-example/shazam-com>